

Timer-as-a-Service

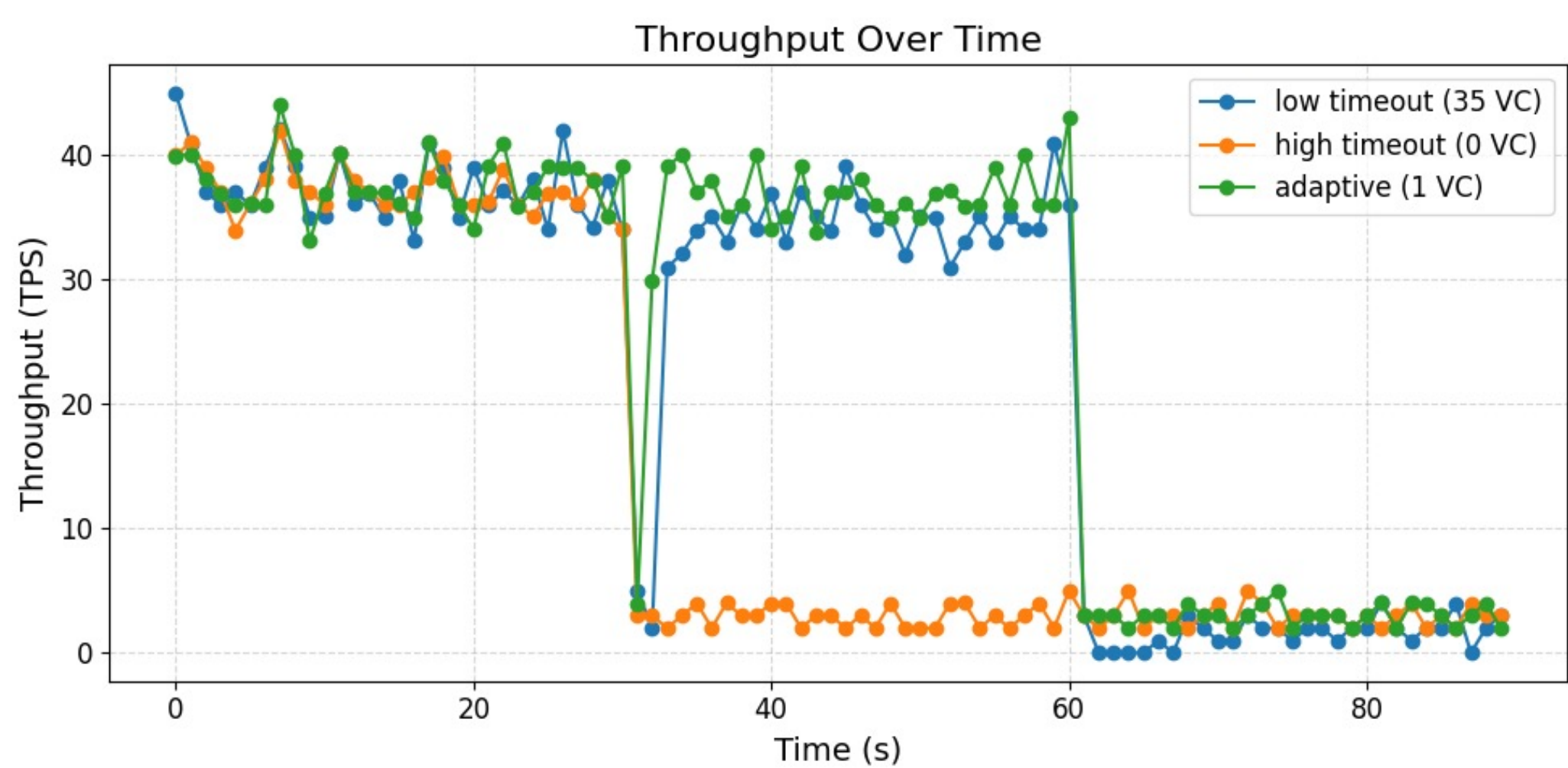
Adaptive Timeout Configuration in Distributed Protocols

Kevin Dharmawan, Prajwal Annigeri, Mohammad Javad Amiri
Stony Brook University

MOTIVATION

A timeout that works well in one regime (e.g., workload, configuration, failure scenario) may cause stalls or instability in another. Static and heuristic configurations are therefore fragile.

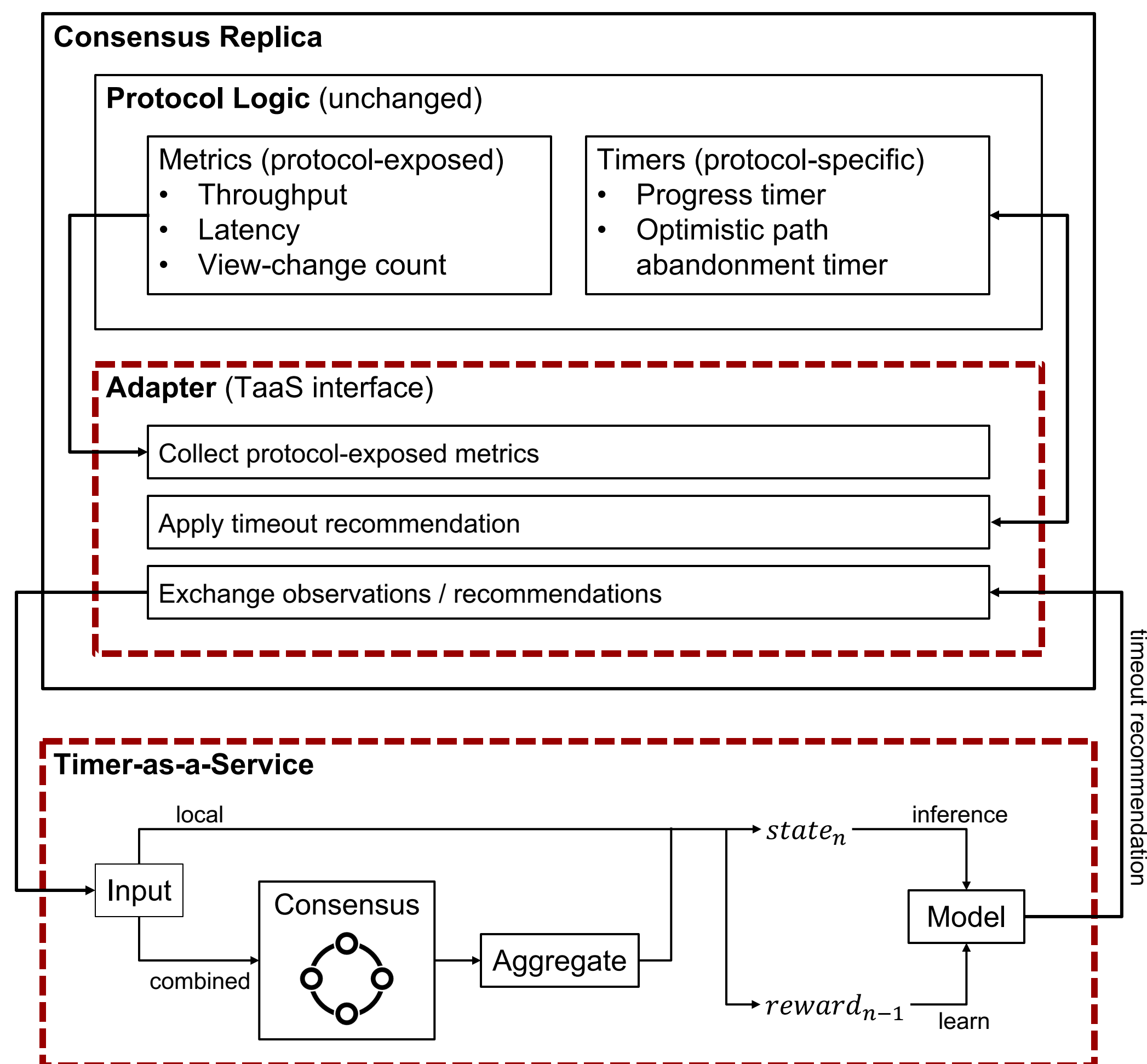
Motivational experiment: Static timeout vs. adaptive timeout under different conditions



KEY IDEA: TIMER-AS-A-SERVICE

- An external, protocol-agnostic timeout recommendation service
- Observes runtime metrics and suggests timeout values
- Improves performance under changing workloads and networks
- Protocol logic and safety remain unchanged

ARCHITECTURE



CORRECTNESS

TaaS preserves protocol correctness by construction:

- Uses identical assumptions as the original protocol
- **Safety:** unchanged and fully enforced by the underlying protocol
- **Liveness:** may regress to baseline behavior but is never violated

TaaS affects only timeout configuration and will not demonstrate any unsafe behaviors.

DATA AGGREGATION

TaaS allows users to configure the following aspects of data aggregation:

- Whether adaptation uses local observations only or aggregated combined observations.
- The aggregation function applied to combined observations.
- The relative weights assigned to local and shared observations.

These choices can be expressed abstractly as:

$$x_t = f(w^{local} \cdot x_t^{local}, w^{shared} \tilde{x}_t^{shared})$$

TIMER CLASSES

- We consider a representative set of timer classes in consensus and commit protocols.
- Each class represents a tunable control knob and is sensitive to different conditions.

The timer classes we consider include:

- Progress / liveness: detect stalled progress and trigger view change.
- Optimistic path abandonment: bound how long the protocol waits under optimistic assumptions.
- Transaction batching: bound batching in multi-phase commit protocols.

The table below summarizes when adapting each timer class can help mitigate different conditions.

Operating Condition	Progress / Liveness	Optimistic Abandonment	Transaction Batching
Latency / delay	✓	✓	✓
Packet loss / dark		✓	
Crashes		✓	
Workload	✓		✓

We model network/failure effects using a **peer-to-peer delay model** (min/max, on/off) and workload effects using **demand and request mix** (e.g., read/write ratio, concurrent clients).